Spring AI — End-to-End Learning & POC Guide

For Java / Spring Boot / Microservices developers
A step-by-step path from a basic Spring AI chat application to RAG, tool calling, microservices integration, and Agentic AI.

1. What is Spring AI?

Spring AI is a Spring ecosystem project that provides application-level abstractions for integrating AI models and AI capabilities into Spring applications. Instead of writing provider-specific integration code everywhere, an application can use Spring AI APIs such as ChatClient and integrations for chat models, embeddings, vector stores, tool calling, advisors and other AI workflows.

2. What We Will Build

```
Client
  |
  | REST API
  v
Spring Boot Application
  |
  v
Spring AI ChatClient
  |
  v
AI Model
  |
  v
AI Response
```

Example request

```
POST /api/ai/chat

{
  "message": "Explain microservices in simple words"
}
```

3. What Can We Achieve?

Level	What it means
Level 1 — AI Chat	Ask questions, generate text, explain concepts, generate Java code, and review code.
Level 2 — Prompt Engineering	Control the role, instructions, format and style of AI responses.
Level 3 — RAG	Connect the model to your own documents and enterprise knowledge using retrieval and vector search.
Level 4 — AI + Database	Expose controlled application operations so AI-assisted workflows can work with business data.
Level 5 — Tool Calling	Allow the model to request specific application functions such as getCustomerDetails or getAccountBalance.
Level 6 — Agentic AI	Build multi-step workflows where an AI system selects tools and coordinates tasks across services.

4. Recommended Technology Stack

```
Java 21
Spring Boot 4.x
Spring AI 2.x
Maven
Spring Web
OpenAI-compatible chat model
PostgreSQL / Vector Store for later RAG stages
Kafka for later event-driven workflows
```

```
Docker for local infrastructure
```

5. Step 1 — Create the Spring Boot Project

Use Spring Initializr. Select Maven, Java, Spring Boot 4.x, Jar packaging and Java 21. Add Spring Web and the Spring AI OpenAI model starter.

6. Project Structure

```
spring-ai-demo
|
+-- src/main/java/com/example/springaidemo
|   |
|   +-- SpringAiDemoApplication.java
|   |
|   +-- controller
|   |   +-- AiController.java
|   |
|   +-- service
|       +-- AiService.java
|
+-- src/main/resources
|   +-- application.properties
|
+-- pom.xml
```

7. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.0</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>spring-ai-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.ai</groupId>
                <artifactId>spring-ai-bom</artifactId>
                <version>2.0.1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-starter-model-openai</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
```

```
            <scope>test</scope>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

8. application.properties

```
spring.application.name=spring-ai-demo

spring.ai.openai.api-key=${OPENAI_API_KEY}

spring.ai.openai.chat.model=gpt-5-mini
```

Security note

Do not hard-code an API key in Java or commit it to Git. Store it as an environment variable or use a secure secret-management mechanism in real environments.

9. Main Application

```
package com.example.springaidemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringAiDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            SpringAiDemoApplication.class,
            args
        );
    }
}
```

10. AI Service

```
package com.example.springaidemo.service;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.stereotype.Service;

@Service
public class AiService {

    private final ChatClient chatClient;

    public AiService(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();
    }

    public String askAI(String message) {
        return chatClient
                .prompt()
                .user(message)
                .call()
                .content();
    }
}
```

How it works

```
REST request
```

```
    |
    v
AiController
    |
    v
AiService
    |
    v
ChatClient
    |
    v
Spring AI
    |
    v
AI Model
    |
    v
Response
```

11. REST Controller

```
package com.example.springaidemo.controller;

import com.example.springaidemo.service.AiService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/ai")
public class AiController {

    private final AiService aiService;

    public AiController(AiService aiService) {
        this.aiService = aiService;
    }

    @GetMapping("/chat")
    public String chat(@RequestParam String message) {
        return aiService.askAI(message);
    }
}
```

12. Run the Application

```
mvn spring-boot:run
```

Or run SpringAiDemoApplication from IntelliJ IDEA or another IDE.

13. Test the API

```
GET http://localhost:8080/api/ai/chat?message=Explain%20microservices
```

The request travels from the REST controller to the service, then through ChatClient and Spring AI to the configured AI model. The generated answer is returned to the client.

14. Phase 2 — Prompt Engineering

Instead of sending only a user message, introduce system instructions that define the AI's role and response rules.

```
You are a senior Java architect.

Explain the following topic:
{question}

Rules:
- Use simple English
- Give a real-world banking example
- Include a Java/Spring Boot example
```

Useful concepts to learn

```
System message  -> defines role/instructions
User message    -> contains the user's request
Prompt template -> reusable prompt structure
Structured output -> predictable application-friendly response
```

15. Phase 3 — Chat Memory

```
User: My name is Jitendra
AI: Nice to meet you.

User: What is my name?
AI: Your name is Jitendra.
```

For real applications, conversation history should be deliberately designed: identify the conversation, control the amount of history, protect sensitive information, and decide how long memory should live.

16. Phase 4 — RAG (Retrieval-Augmented Generation)

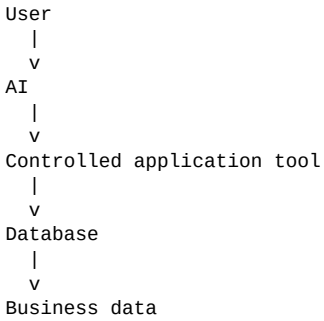
```
PDF / Docs / FAQs
       |
       v
Document Loader
       |
       v
Chunking
       |
       v
Embeddings
       |
       v
Vector Database
       |
       v
Retriever
       |
       v
Relevant context
       |
       v
LLM
       |
       v
Answer
```

RAG is useful when answers should be grounded in your own enterprise information. Typical sources include API documentation, architecture documents, product manuals, policies, FAQs and technical documents.

Example questions

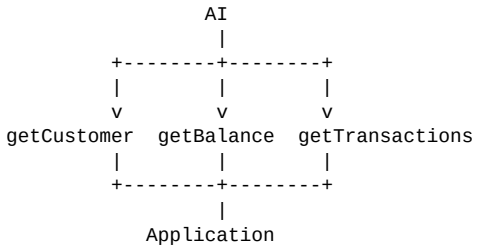
```
What is the payment timeout?
What happens if payment-service fails?
What is the retry policy?
Explain this API specification.
```

17. Phase 5 — AI + Database



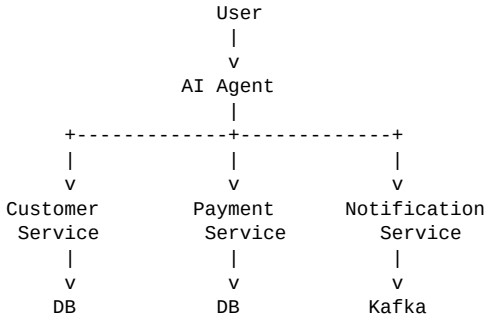
The safer enterprise pattern is to expose controlled business operations rather than giving an AI model unrestricted database access.

18. Phase 6 — Tool Calling



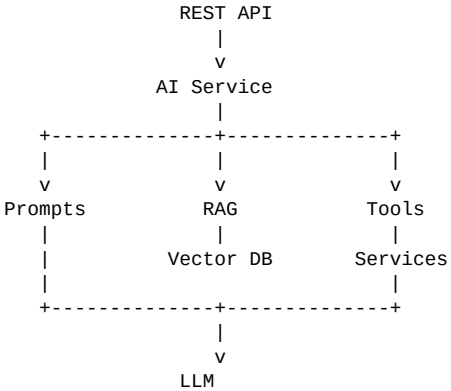
Example tools in a banking POC could include getCustomerDetails(), getAccountBalance(), getTransactionHistory(), createPayment() and cancelPayment(). In production, each operation must have strong authorization, validation, auditability and business controls.

19. Phase 7 — Agentic AI



A multi-step workflow might be: validate customer → check balance → validate beneficiary → execute payment → check status → notify customer. The agentic layer can select tools and coordinate steps, while business services remain responsible for actual authorization and transaction rules.

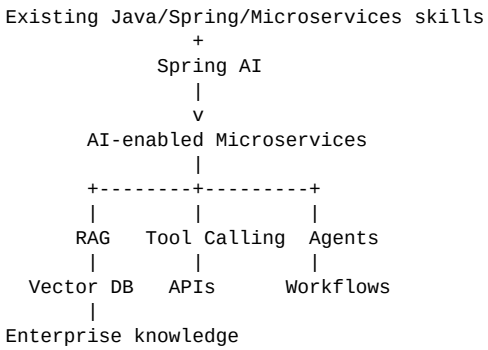
20. Enterprise Architecture Direction



21. Recommended Learning Roadmap

1. Step 1 — Basic Spring AI Chat
2. Step 2 — System + User Prompts
3. Step 3 — Structured Output
4. Step 4 — Chat Memory
5. Step 5 — RAG + PDF
6. Step 6 — PostgreSQL + Vector Database
7. Step 7 — Tool Calling
8. Step 8 — Multiple Tools
9. Step 9 — AI + Microservices
10. Step 10 — Agentic AI
11. Step 11 — Kafka + Async Processing
12. Step 12 — Security + Observability
13. Step 13 — Docker + Deployment

22. How This Maps to Your Existing Skills



This progression lets a Java/Spring developer move from basic AI integration to enterprise AI patterns without abandoning familiar architectural principles such as APIs, service boundaries, security, observability, asynchronous messaging and databases.

23. Interview-Level Summary

Spring AI: A Spring ecosystem project that provides abstractions and integrations for building AI-enabled applications. The key learning areas for an experienced Spring Boot developer are ChatClient, prompts, model integration, memory, RAG, embeddings, vector stores, tool calling and agentic workflows.

Practical project goal: Build a banking-oriented AI POC that starts with a chat endpoint, then adds enterprise documents through RAG, controlled microservice tools, Kafka-based events, security and observability, and finally a multi-step agentic workflow.

24. Important Production Considerations

• Never expose API keys in source code or Git repositories.

• Authenticate and authorize every business operation.

• Do not allow unrestricted AI access to production databases.

• Validate tool inputs and outputs.

• Add audit logs for sensitive actions.

• Apply timeouts, retries and circuit breakers where appropriate.

• Control prompt and response sizes to manage cost and latency.

• Monitor model calls, errors, latency and token usage.

• Treat retrieved documents as untrusted input and apply appropriate security controls.

• Use human approval for high-impact or irreversible business operations when required.

Note: Spring AI and model-provider configuration changes over time. Before starting a new project, verify the exact Spring Boot, Spring AI starter, model name and provider configuration against the current official documentation.